New issue

Jump to bottom

Feature request: Add argument "fill" to lineplot() #2410

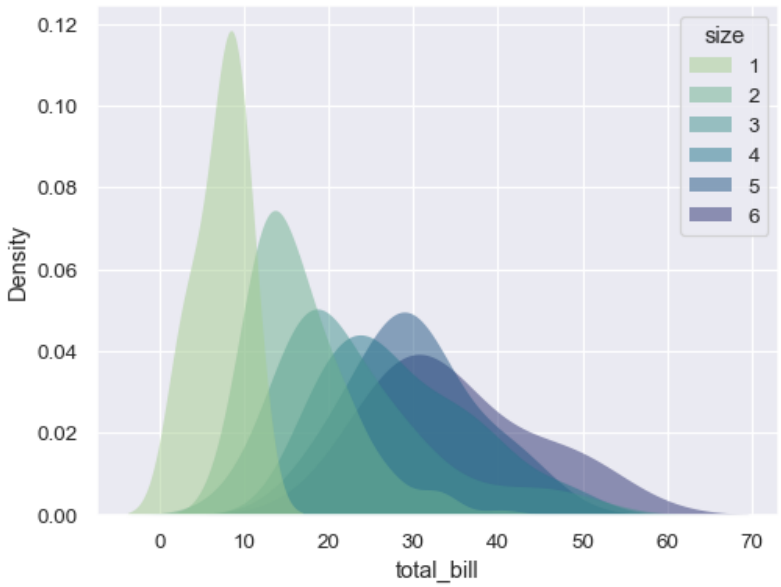
Closed **normanius** opened this issue on Jan 1, 2021 · 8 comments

Labels wishlist

normanius commented on Jan 1, 2021 • edited

kdeplot() offers an argument fill. It would be nice to offer this argument to lineplot() as well.

```
sns.lineplot(
   data=data, x="x", y="y", hue="category", fill=True, palette="crest", alpha=.5, linewidth=0
)
```



(image from kdeplot docu)

Keep in mind that the lineplot can be used in the context of a polar plot (axis projection: "polar").

PS: happy New Year!

2 2

mwaskom commented on Jan 1, 2021 Owner

I am -1 on adding this to `lineplot`, but am somewhat open to the idea of an `areaplot`. There are a few things one would need to think about though:

- There would need to be a way to show errorbars, I'm not really sure that there's a good way to do that with area plots
- Currently, the way things work in seaborn is that all of the plots in the same module (and same figure-level interface) support the same semantic mappings. `lineplot` and `scatterplot` support `hue`, `size`, and `style`. It's not obvious how to add `size` and `style`. Well,

- Naturally the next step after giving the mouse the `areaplot` cookie is that they would want some "stacked" area plot milk. The logic that does stacking for filled densities or histograms assumes a different internal representation of the data than is in the relational plots. On a medium-long term time horizon, I'd like to make that more general, but I'd consider that a prerequisite to adding this.

So if the question is "should seaborn have an area plot that can't do aggregation/errorbars and only has `hue` semantic mapping" my answer would probably be no because that exists in pandas builtin plotting and I try not to devote effort to straight up duplication of plots you can already make with matplotlib or pandas. If some of these problems can be solved, I'd consider it worth adding to seaborn. But it's a low priority right now.

1 1

mwaskom added the wishlist label on Jan 1, 2021

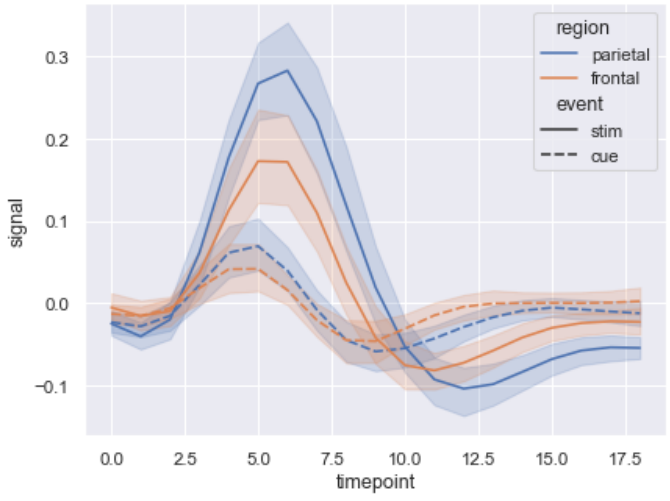
normanius commented on Jan 2, 2021 Author

Thanks. I'm a visualization novice, but I dare to answer your thoughts :)

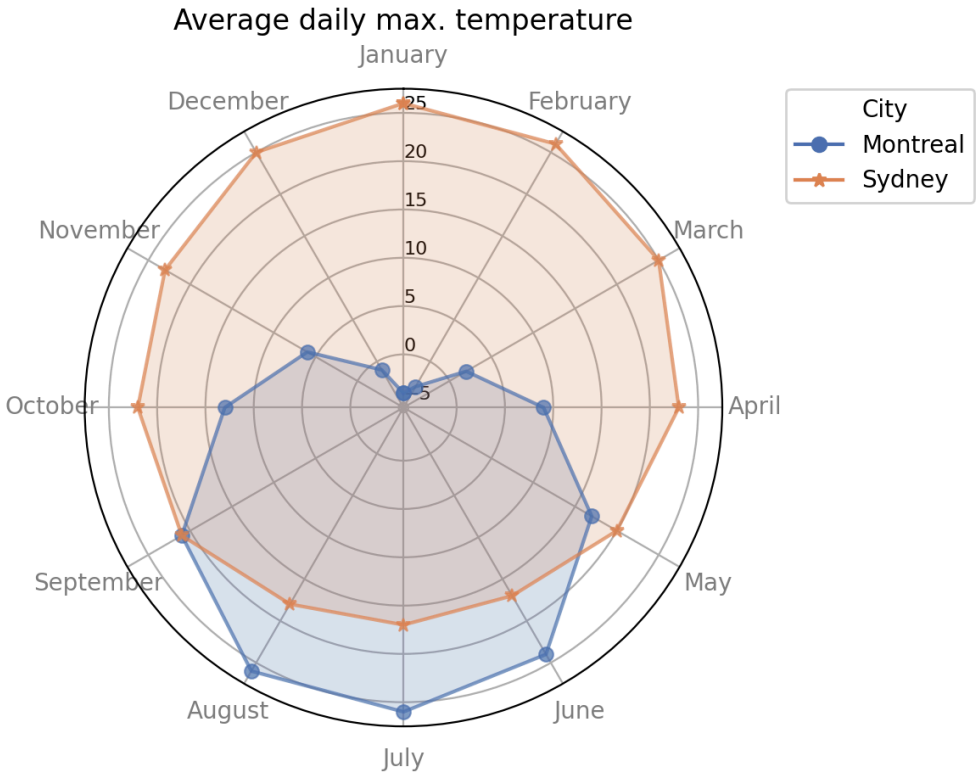
- I wasn't aware of pandas' area-plot feature. Fair point regarding unnecessary code duplication. But isn't seaborn's theme, hue/style selection or handling of the legend superior compared to pandas' plotting features?
- Following your thoughts, I'm not sure if `areaplot()` would really add a value other than just duplicating `DataFrame.plot.area()`. I'm not aware of any means to efficiently visualize uncertainty bounds for `areaplot()` - except the error bars or shaded bounds (see below), which reduces `areaplot()` into a `lineplot(..., fill = <option>)`
- The only "plus" would be the option to create a `stackplot()`. AFAICS this feature is not available in seaborn yet. But since a `stackplot()` essentially is just another `lineplot()` with some data-preprocessing, I'm tempted to say that this additional features doesn't back up the need for an independent areaplot().
- Adding the curve filling adds another type of artist with properties that differ from those of the line (color, alpha), or entirely different ones (like hatching). To match/mimic seaborn semantics, I'd simply use hue for both line and fill polygon, style and size would apply only to the line. I'd introduce, however, separate arguments for `fillalpha` or `fillcolor` (if it should be set to fixed values).

Summary: After considering your arguments, I don't really see added value for an independent `areaplot()` other than duplicating `pd.DataFrame.plot.area()` with seaborn semantics (which I consider desirable). I think, the fill polygon is an attribute of a line and therefore should be part of `lineplot` rather than devoting a separate function to it. But I see the problems regarding seaborn semantics.

Example: Error bounds drawn with `lineplot()`:



Example: Where area filling can be useful



mwaskom commented on Jan 2, 2021 Owner

> But isn't seaborn's theme, hue/style selection or handling of the legend superior compared to pandas' plotting features?

Not really, because seaborn does all of its theming through the matplotlib rc system, which pandas plots also pick up. So if you set a theme "in seaborn", it will affect pandas plots too.

I don't think it makes much sense to have a `lineplot` that draws areas ... `lineplot` uses filling to represent uncertainty which is different (in tension with, actually) using it to represent the quantity.

But ... on the user side it's easy to define a simple function that adds fills to a line plot, e.g. something like

```
def fill_under_lines(ax=None, alpha=.2, **kwargs):
    if ax is None:
        ax = plt.gca()
    for line in ax.lines:
        x, y = line.get_xydata().T
        ax.fill_between(x, 0, y, color=line.get_color(), alpha=alpha, **kwargs)
```

If you have that sitting in a personal function library, you can call it after you make a lineplot and it will add fills that match the lines in your plot, essentially just as you'd like.

4

normanius commented on Jan 2, 2021 • edited Author

> But ... on the user side it's easy to define a simple function that adds fills to a line plot, e.g. something like

Below an example for "hybrid" plotting using seaborn and pandas. Maybe this will be of use to anyone.

Either way, there are convincing arguments now to drop this feature. The only question that remains is why `fill` was introduced to `kdeplot()` :)

Example: Hybrid plotting with seaborn and pandas

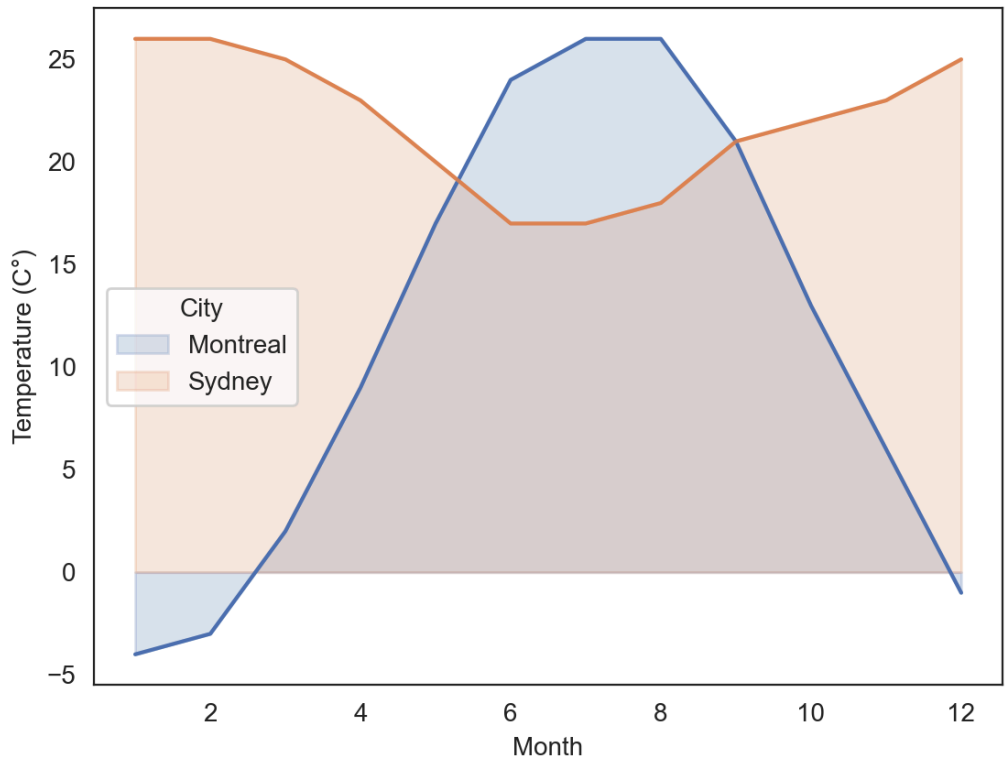
```
sns.set_style(style="white")
sns.set_palette(palette="deep")
_, ax = plt.subplots()

# df is in long format, pandas requires wide format.
df = pd.read_csv("data.csv")
df_wide = df.pivot(index="Month", columns="City", values="Value")

sns.lineplot(x="Month", y="Value", hue="City", data=df, ax=ax, legend=False)
ax.set_prop_cycle(None)
df_wide.plot.area(stacked=False, alpha=0.2, ax=ax)
ax.set_xlabel("Month")
ax.set_ylabel("Temperature (C°)")
plt.show()
```

And for completeness the dataframe: data.csv.zip

The result:



2

mwaskom commented on Jan 3, 2021 • edited Owner

magnitude, where zero and ratios are meaningful. In that sense, your temperature plot is kind of a counterexample: 0 celsius has a quasi-meaningful physical definition, but it doesn't make sense to say that 20deg is "twice as warm" as 10deg, or that two months of 10deg weather are comparable to one month of 20 deg weather.

And so we can answer:

> The only question that remains is why fill was introduced to kdeplot

`kdeplot` is not just a fancy line plot, it's a graphical representation of a probability distribution. Probability density checks the boxes above (zero and ratios are meaningful) and *areas* in a kdeplot are actually super-meaningful, in the sense that "density" is not a directly interpretable measure but that the area under a density curve corresponds to probability. So it's a natural fit there.

normanius commented on Jan 3, 2021 • edited — Author

Sorry, the last (rhetorical) question was probably unnecessary. I know that kdeplot() does way more than just plotting lines and that in the case of probability density functions the area has a meaning. But when it comes to plotting lines, there's a "fill" option in kdeplot(). This option is missing in other tools like lineplot(), which I felt was a bit inconsistent. But as discussed above, I now understand that there're alternatives and reasons why areaplots (as I conceived it) are not urgently needed in seaborn.

As I see it, the filling has (at least) two functions: 1) representation of information (as the area under the kde), 2) representation of entity / graphical emphasize. What applies depends on the use case. I used the temperature example because I had the data at hand for a quick demo. I agree that in this case, the fill neither serves 1) or 2). The earlier example with polar coordinates at least illustrates function 2) of the fill.

mattdm commented on Jan 26, 2022

For whatever it's worth,

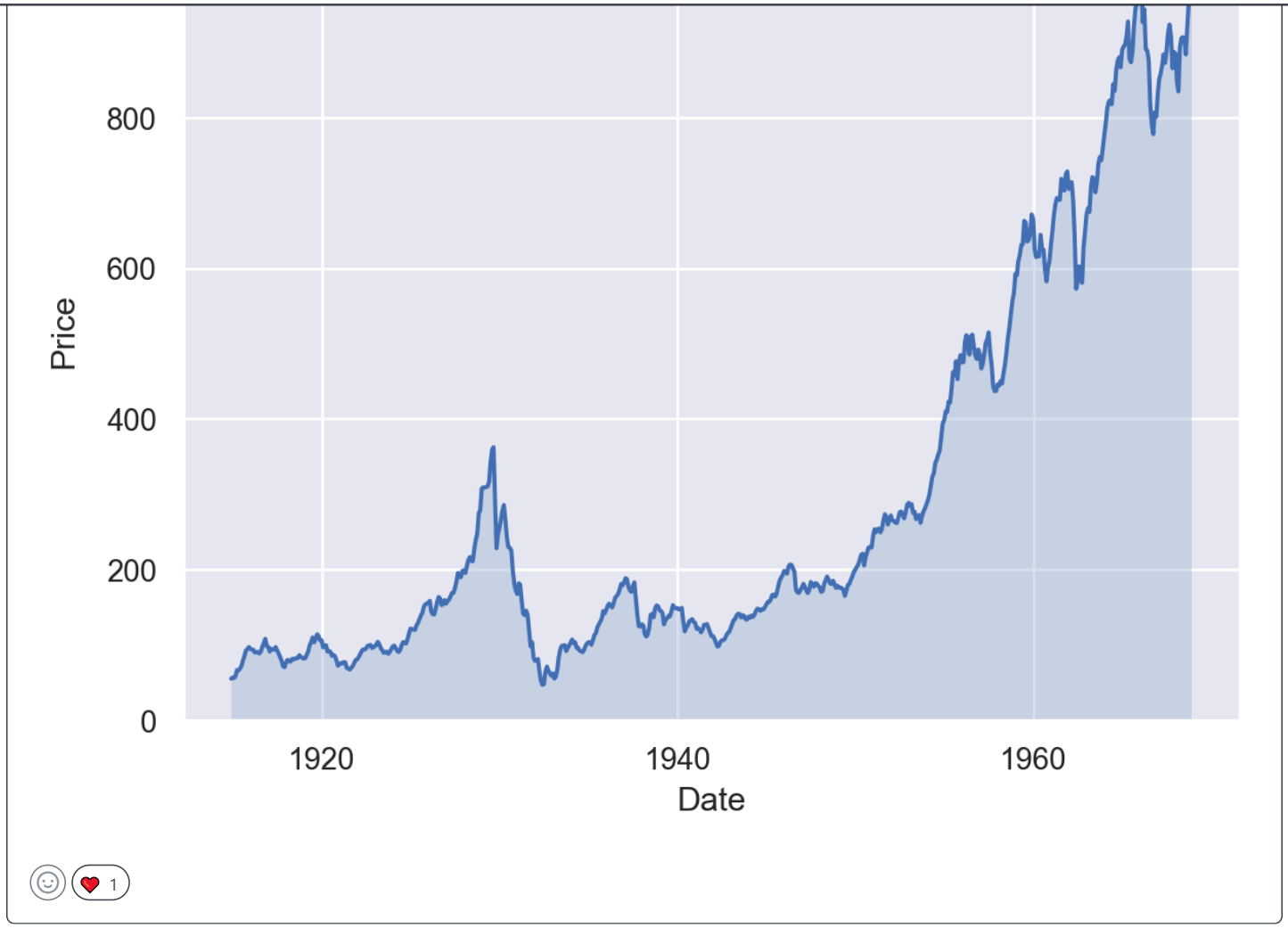
```
# df is in long format, pandas requires wide format.
```

... is the reason I'd really like this. I know how to use pivot, but it'd be nice to just be consistent.

mwaskom commented on Sep 14, 2022 — Owner

With the `Area` mark added to the new objects interface, I'm going to close this as complete. It's not impossible that something like `areaplot` would get added to the plotting functions, but it's not currently planned and doesn't seem like a great fit for the reasons enumerated above.

```
dowjones = sns.load_dataset("dowjones")
so.Plot(dowjones, "Date", "Price").add(so.Line()).add(so.Area(edgewidth=0))
```



1

mwaskom closed this as completed on Sep 14, 2022

mwaskom mentioned this issue on Dec 6, 2022

pandas plotting backend? #3176

Closed

Assignees

No one assigned

Labels

wishlist

Projects

None yet

Milestone

No milestone

3 participants